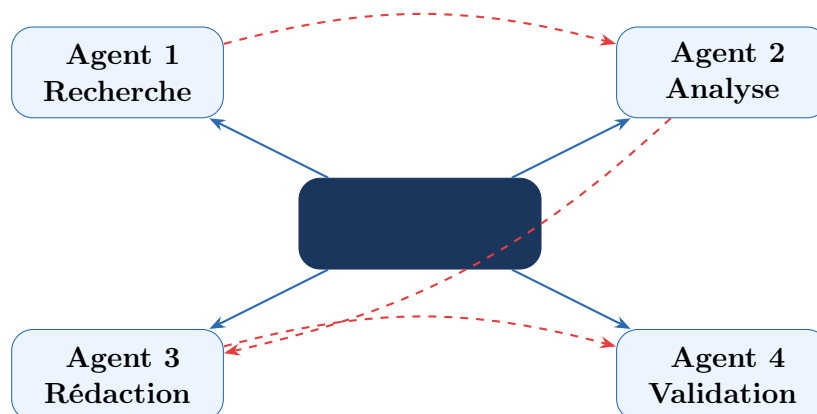


Cahier des Charges – Projet

Contrôle Continu

IA Distribuée & Systèmes Multi-Agents

Construction d'un Système Multi-Agents avec RAG & Orchestration LangChain



- ◇ **Durée :** 15 jours
- ◇ **Modalité :** Binôme
- ◇ **Évaluation :** Soutenance + Questions sur le code
- ◇ **Technologies :** LangChain, LlamaIndex, LLMs

Table des matières

1	Contexte et Objectifs	2
1.1	Contexte Pédagogique	2
1.2	Objectifs du Projet	2
2	Spécifications Techniques	2
2.1	Architecture Générale	2
2.2	Stack Technologique Obligatoire	3
2.3	Composants Requis	3
2.3.1	RAG via LlamaIndex	3
2.3.2	Agents LangChain	3
2.3.3	Orchestration	4
3	Exemples de Systèmes Multi-Agents	4
3.1	Exemple 1 : Assistant de Recherche Académique	4
3.2	Exemple 2 : Support Client Intelligent	4
3.3	Exemple 3 : Analyseur Financier d'Entreprise	5
3.4	Exemple 4 : Assistant Juridique	5
4	Livrables Attendus	6
4.1	Code Source	6
4.2	Documentation Technique	6
4.3	Démonstration	6
5	Modalités d'Évaluation	7
5.1	Barème	7
5.2	Soutenance & Questions sur le Code	7
6	Ressources & Conseils	7
6.1	Ressources Recommandées	8
6.2	Conseils Pratiques	8
7	Planning Indicatif	8

Contexte et Objectifs

Contexte Pédagogique

Ce projet s'inscrit dans le cadre du module “**IA Distribuée et Systèmes Multi-Agents**”. Il constitue le **contrôle continu** du module et vise à évaluer votre capacité à concevoir, implémenter et orchestrer un système multi-agents intelligent exploitant des modèles de langage (LLMs) et des données privées.

Objectifs du Projet

Les objectifs principaux sont :

1. **Concevoir** un système multi-agents avec un cas d'usage concret et pertinent.
2. **Implémenter** la technologie **RAG** (Retrieval-Augmented Generation) via **LlamaIndex** pour contextualiser les agents avec des données privées.
3. **Développer** les agents IA, leurs outils et l'orchestration via **LangChain**.
4. **Démontrer** la collaboration inter-agents et la valeur ajoutée du système distribué.

△ Contraintes Obligatoires

- Le projet est réalisé en **binôme** (pas de trinôme, pas de travail individuel, sauf cas de force majeure.).
- Durée : **15 jours** à compter de la date de distribution.
- L'utilisation de **LlamaIndex** (pour le RAG) et de **LangChain** (pour les agents) est **obligatoire**.
- Chaque membre du binôme doit maîtriser **l'intégralité du code**.

Spécifications Techniques

Architecture Générale

Votre système doit respecter l'architecture suivante :

Stack Technologique Obligatoire

Technologie	Rôle	Statut
LangChain	Développement des agents, définition des outils (tools), orchestration et chaînes de raisonnement	Obligatoire
LlamaIndex	Indexation des données privées, pipeline RAG, query engine pour la contextualisation	Obligatoire
LLM Provider	Au choix : Ollama (modèles open source locaux), API Anthropic (Claude), API OpenAI (GPT), API Groq, ou autre	Obligatoire
Python	Langage de développement	Obligatoire
Vector Store	ChromaDB, FAISS, Pinecone, ou autre	Recommandé

Composants Requis

RAG via LlamaIndex

Vous devez implémenter un pipeline RAG complet permettant à vos agents d'accéder à des **données privées** que le modèle de langage ne connaît pas nativement :

- **Ingestion** : chargement de documents (PDF, CSV, TXT, JSON, pages web, etc.)
- **Indexation** : découpage en chunks, génération d'embeddings, stockage vectoriel.
- **Retrieval** : recherche sémantique pour extraire les passages pertinents.
- **Augmentation** : injection du contexte récupéré dans le prompt de l'agent.

Agents LangChain

Chaque agent doit avoir :

- Un **rôle** clairement défini (persona / system prompt).

- Des **outils** (tools) spécifiques à sa mission.
- Une capacité de **raisonnement** (ReAct, chain-of-thought, etc.).
- La possibilité d'**interagir** avec d'autres agents ou de déléguer des tâches.

Orchestration

Le système doit démontrer une **orchestration** claire :

- Flux de travail défini (séquentiel, parallèle, ou conditionnel).
- Gestion de l'état partagé entre agents.
- Mécanisme de routage ou de délégation des tâches.

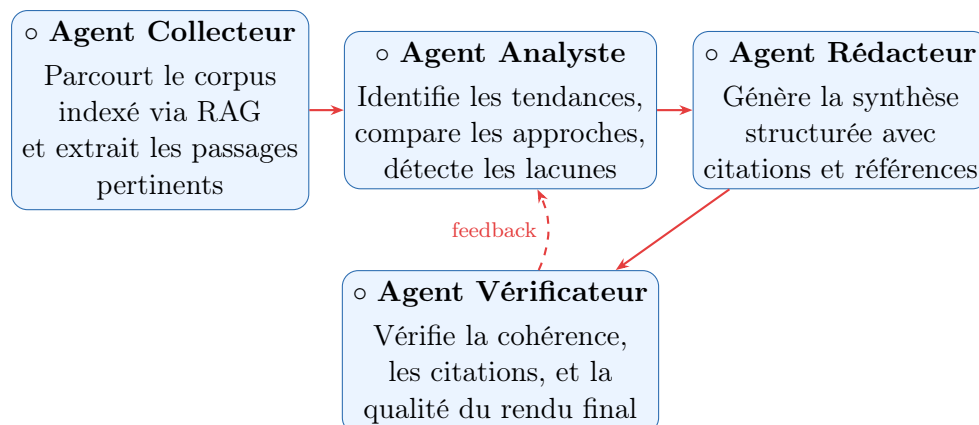
Exemples de Systèmes Multi-Agents

Voici plusieurs exemples concrets pour vous inspirer. **Vous êtes libres de proposer votre propre cas d'usage**, à condition qu'il respecte les contraintes techniques.

Exemple 1 : Assistant de Recherche Académique

◦ Cas d'usage : Recherche & Synthèse Documentaire

Un système qui aide un chercheur à analyser un corpus de publications scientifiques (données privées) pour produire un état de l'art structuré.



Données privées (RAG) : corpus de 50+ articles PDF indexés via LlamaIndex.

Exemple 2 : Support Client Intelligent

◦ Cas d'usage : Service Client Multi-Niveaux

Un système de support client qui traite les demandes entrantes en s'appuyant sur la documentation interne de l'entreprise (données privées).

Agent	Rôle	Outils
Agent Routeur	Analyse la demande client et la dirige vers l'agent spécialisé approprié	Classificateur de requêtes, détecteur de sentiment
Agent FAQ	Répond aux questions fréquentes en consultant la base de connaissances interne	RAG sur la documentation produit, historique FAQ
Agent Technique	Résout les problèmes techniques en consultant les guides de dépannage	RAG sur les manuels techniques, logs système
Agent Escalade	Prépare un résumé complet pour le support humain si non résolu	Générateur de tickets, résumeur de conversation

Données privées (RAG) : documentation produit, guides techniques, FAQ internes, historique de tickets.

Exemple 3 : Analyseur Financier d'Entreprise

◦ Cas d'usage : Analyse Financière Automatisée

Un système qui analyse les rapports financiers internes d'une entreprise pour générer des insights et des recommandations.

Les 4 agents collaborent ainsi :

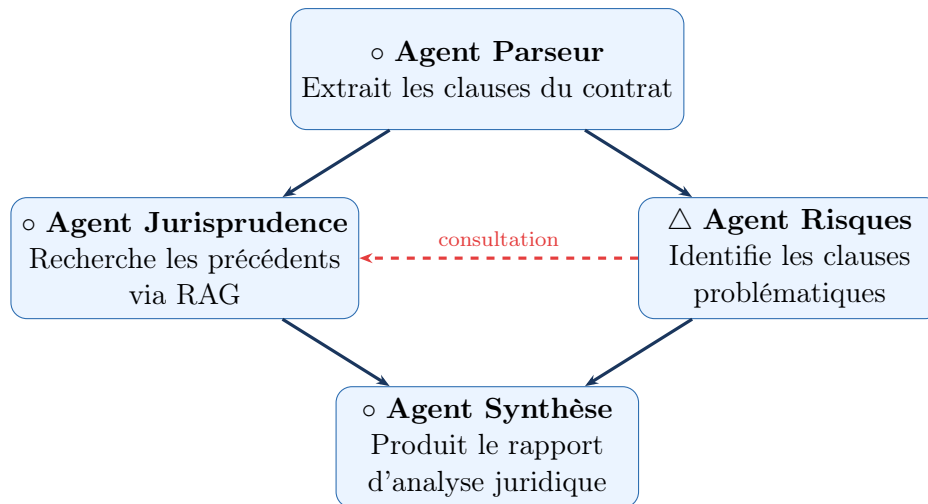
- Étape 1** → **Agent Extracteur** : parse les rapports financiers (PDF/Excel) via RAG et extrait les KPIs clés (chiffre d'affaires, marges, dettes, etc.).
- Étape 2** → **Agent Comparateur** : compare les KPIs avec les périodes précédentes et les benchmarks du secteur, identifie les écarts significatifs.
- Étape 3** → **Agent Stratège** : interprète les tendances, identifie les risques et opportunités, propose des recommandations stratégiques.
- Étape 4** → **Agent Rapporteur** : compile le tout dans un rapport exécutif structuré avec visualisations et résumé décisionnel.

Données privées (RAG) : bilans comptables, rapports trimestriels, données de benchmark sectoriels.

Exemple 4 : Assistant Juridique

◦ Cas d'usage : Analyse Contractuelle

Un système qui aide un cabinet juridique à analyser des contrats et identifier les risques potentiels en s'appuyant sur un corpus de jurisprudence privé.



Données privées (RAG) : corpus de jurisprudence, contrats précédents, textes de loi, notes internes du cabinet.

Livrables Attendus

Code Source

• Livrable Principal

- Code source complet et fonctionnel sur un **dépôt Git** (GitHub/GitLab).
- Fichier `README.md` détaillé : description du projet, architecture, instructions d'installation et d'exécution.
- Fichier `requirements.txt` ou `pyproject.toml` avec toutes les dépendances.
- Fichier `.env.example` pour les clés API (sans les vraies clés).
- Données privées de test (ou script pour les générer).

Documentation Technique

Un document (dans le README ou séparé) décrivant :

- Le **cas d'usage** choisi et sa justification.
- L'**architecture** du système (diagramme des agents et leurs interactions).
- Le **pipeline RAG** : type de données, stratégie de chunking, modèle d'embedding, vector store utilisé.
- La description de chaque **agent** : rôle, outils, prompts.
- Le **flux d'orchestration** : comment les agents collaborent.

Démonstration

Une démonstration fonctionnelle montrant :

- L'ingestion de données privées et l'indexation RAG.

- Le fonctionnement de chaque agent individuellement.
- La collaboration entre agents sur un scénario complet.
- La pertinence des réponses grâce au RAG (vs. sans RAG).

Modalités d'Évaluation

Barème

#	Critère d'Évaluation	Pondération
1	Pipeline RAG (LlamaIndex) : qualité de l'ingestion, du chunking, de l'indexation et du retrieval	25%
2	Agents & Outils (LangChain) : pertinence des agents, qualité des outils, prompts bien conçus	25%
3	Orchestration : flux de travail, communication inter-agents, gestion de l'état	20%
4	Cas d'usage & pertinence : originalité, complexité, valeur ajoutée démontrée	15%
5	Qualité du code & documentation : propreté, modularité, README, commentaires	15%

Soutenance & Questions sur le Code

△ Évaluation Individuelle

Lors de la soutenance, **des questions individuelles sur le code seront posées** à chaque membre du binôme. Ces questions porteront sur :

- La compréhension du pipeline RAG : comment fonctionne l'indexation ? Pourquoi ce chunking ?
- La logique des agents : expliquer le raisonnement d'un agent, ses outils, son prompt.
- L'orchestration : comment les agents communiquent-ils ? Comment l'état est-il géré ?
- Les choix techniques : pourquoi tel modèle ? Tel vector store ? Telle stratégie ?

⇒ **Chaque étudiant doit être capable d'expliquer et de justifier n'importe quelle partie du code. Une incapacité à répondre entraînera une pénalité sur la note individuelle.**

Ressources & Conseils

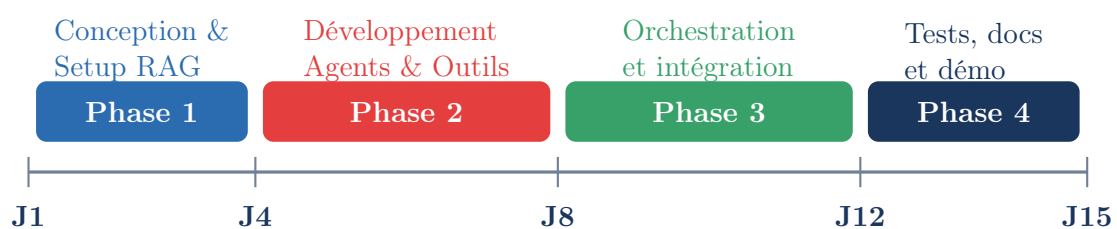
Ressources Recommandées

Ressource	Lien / Description
LangChain Docs	https://python.langchain.com/docs/
LlamaIndex Docs	https://docs.llamaindex.ai/
Ollama	https://ollama.ai/ – modèles open source en local
Anthropic API	https://docs.anthropic.com/ – Claude API
OpenAI API	https://platform.openai.com/docs/
Groq API	https://console.groq.com/docs/ – inférence rapide
ChromaDB	https://www.trychroma.com/ – vector store
FAISS	https://github.com/facebookresearch/faiss

Conseils Pratiques

- ★ **Commencez par le RAG** : faites fonctionner l'ingestion et le retrieval avant de construire les agents.
- ★ **Testez chaque agent isolément** avant de les orchestrer ensemble.
- ★ **Utilisez des données réalistes** : le RAG n'a d'intérêt que si les données privées sont pertinentes pour votre cas d'usage.
- ★ **Versionnez votre code** dès le début avec Git – commits réguliers.
- ★ **Documentez au fur et à mesure**, pas à la fin.
- ★ Si vous utilisez des API payantes, **gérez vos crédits** prudemment. Ollama est gratuit et suffisant pour le développement.
- ★ **Préparez une démo convaincante** : montrez un scénario complet de bout en bout.

Planning Indicatif



Bon courage et bon développement !

N'hésitez pas à poser vos questions.